

Assignment Day 2

- 1) Create a function that returns the square of a number.
- 2) Create a function that returns the cube of a number.
- 3) Create a function that checks whether a number is positive, negative, or zero.
- 4) Create a function that returns the average of three numbers.
- 5) Create a function with optional parameter to calculate salary after tax (default tax = 15%).
- 6) Create a function that returns the minimum value in an array.
- 7) Create a function that counts how many even numbers exist in an array.
- 8) Create a function that reverses a string.
- 9) Create a function that checks if a string is a palindrome.
- 10) Create a function that calculates the area of a triangle using optional parameter for height (default height = 1).
- 11) Create a function that swaps two numbers using ref.
- 12) Create a function that doubles a number using ref.
- 13) Create a function that returns quotient and remainder using out parameters.
- 14) Create a function that calculates rectangle perimeter using out parameter.
- 15) Create a function that accepts an in parameter and explain why it cannot be modified.
- 16) Create a struct Student with fields: Id, Name, Grade and a constructor to initialize them.
- 17) Add a method inside Student struct that returns pass/fail based on Grade.
- 18) Create a struct BankAccount with fields: AccountNumber and Balance and a constructor.
- 19) Add methods Deposit(ref double amount) and Withdraw(ref double amount) inside BankAccount.
- 20) Create a struct Order with fields: ProductName, Price, Quantity and constructor. Add method GetTotal().

Challenge Question:

- 21) Create a mini system that manages a list of BankAccount structs:
 - Create at least 3 accounts.
 - Implement a function to transfer money between two accounts using ref.
 - Use out parameter to return transaction status (Success/Failed).
 - Use optional parameter to apply transaction fee (default = 2%).
 - Print final balances of all accounts.
- 22) Create a function that returns the maximum value in an array.
- 23) Create a function that sorts an array in ascending order without using built-in sort functions.
- 24) Create a function that removes duplicate values from an array and returns a new array.
- 25) Write a function that searches for a specific number inside an array and returns its index (-1 if not found).

Submission Requirements:

- Clean and readable code.
- Proper function calling.
- Correct use of optional parameters.
- Correct use of ref, out, and in.
- All structs must contain constructors.